

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Ingeniería en Ciencia y Sistemas
Sistemas Operativos 1
Ing. Otto Soto



Hospital

Sergio Alejandro Rodriguez López

202131443

01 de mayo de 2026

1. Introducción

El sistema desarrollado es una simulación concurrente de un departamento de emergencias de un hospital, programado en Java (con Spring Boot) para el backend y Vue.js para el frontend. El objetivo principal es gestionar la llegada concurrente de múltiples pacientes, quienes requieren distintos tipos y cantidades de recursos médicos según su nivel de triaje, todo bajo un ambiente estrictamente seguro para la concurrencia que previene condiciones de carrera e interbloqueos no deseados.

2. Decisiones de Diseño y Arquitectura Concurrente

Para satisfacer los requerimientos del enunciado, se diseñó un modelo concurrente donde cada Paciente actúa como un hilo independiente (Thread / Runnable). Esto simula con gran fidelidad la vida real, donde los pacientes actúan de forma asíncrona.

El componente central que coordina estos hilos es el GestorRecursos. Se decidió utilizar un gestor centralizado para evitar que la lógica de adquisición de recursos se esparciera por toda la aplicación, reduciendo la posibilidad de errores y fugas de recursos.

2.1 Exclusión Mutua y Sincronización

Se utilizó un mecanismo de ReentrantLock (lockAsignacion) con propiedad *fair* (justa) en el GestorRecursos al momento de evaluar y procesar la cola de espera. Justificación:

- Exclusión mutua: Es imprescindible que cuando el sistema evalúa si hay recursos suficientes y procede a asignarlos a un paciente, ningún otro hilo modifique el estado de los recursos.
- Consistencia: Si no existiera este bloqueo, múltiples hilos podrían evaluar la condición `hayRecursosDisponibles == true` simultáneamente e intentar extraer recursos, causando que los conteos sean erróneos e incluso que caigan en valores negativos.
- Eficiencia: El uso de ReentrantLock permite manejar el vaciado y procesamiento de la cola de manera eficiente sin bloquear todo el objeto (como ocurriría si se usara `synchronized` a nivel de clase).

2.2 Semáforos (Semaphore) para Gestión de Recursos

Los recursos médicos del hospital (salas, quirófanos, médicos, cirujanos, enfermeras, ventiladores y monitores) son limitados y su cantidad está bien definida. Para administrarlos, se implementaron Semáforos inicializados con su capacidad máxima (ej. `new Semaphore(10, true)` para las salas). Justificación:

- Los semáforos son la primitiva ideal para este problema, ya que permiten contabilizar y limitar el acceso a un número finito de recursos idénticos.
- Se crearon semáforos *fair* (`true`) para garantizar que el hilo que lleva más tiempo esperando obtenga prioridad cuando los permisos sean liberados a nivel del sistema operativo.

- El método `acquire()` garantiza la reserva segura del recurso, y `release()` lo devuelve, de forma atómica y sin necesidad de escribir código manual de contadores compartidos, que sería más propenso a errores.

2.3 Cola de Espera (PriorityBlockingQueue)

Para almacenar a los pacientes que no pueden ser atendidos inmediatamente, se decidió utilizar una `PriorityBlockingQueue`. Justificación:

- Es una estructura `thread-safe` (segura para hilos) que provee ordenamiento automático.
- El orden se establece a través de la interfaz `Comparable` implementada en la clase `Paciente`, garantizando que la prioridad de triaje (Nivel 1 sobre Nivel 5) y el orden de llegada (ID de paciente) determinen quién sale primero de la cola.
- Al ser *Blocking*, facilita operaciones concurrentes seguras al ingresar y drenar elementos durante la evaluación de asignación.

2.4 Prevención de Condiciones de Bloqueo Permanentes en la Cola

Se identificó y corrigió un problema algorítmico potencial (inanición): si el primer paciente en la cola requiere muchos recursos (ej. Quirófano) y no los hay, un sistema ingenuo bloquearía el procesamiento. Se implementó una lógica donde la cola se drena por completo. Si un paciente requiere recursos no disponibles, se evalúa al siguiente. Los pacientes no atendidos se agrupan en una lista temporal y se reinsertan en la cola al finalizar el ciclo. Esto maximiza la utilización de recursos, permitiendo que pacientes menos críticos utilicen recursos libres si no interfieren con la atención de los más críticos.

3. Demostración y Manejo de Deadlock

Para fines educativos y de calificación, se implementó un modo especial para inducir y visualizar un Deadlock (Interbloqueo). En el caso simulado:

- El Paciente A adquiere a un Cirujano y se queda esperando un Ventilador.
- El Paciente B adquiere un Ventilador y se queda esperando a un Cirujano. Ambos se bloquean mutuamente, ilustrando el problema de retención y espera circular.

Manejo: Se utiliza `tryAcquire(tiempo)` con `timeout` en lugar de un `acquire()` infinito. Al detectar que se agotó el tiempo, el sistema identifica el interbloqueo y notifica al frontend. Se pausa la simulación de ambos hilos usando un `CountDownLatch`, quedando a la espera de que el administrador (vía frontend) elija qué paciente será favorecido. El paciente elegido obtiene sus recursos y avanza, mientras que el otro retrocede, liberando su recurso parcial y regresando a la cola, eliminando así la condición de retención y espera, y resolviendo el deadlock.

4. Comunicación Asíncrona con el Frontend

Se optó por utilizar STOMP sobre WebSockets para enviar métricas y datos en tiempo real al cliente web. Esto era preferible a un modelo de *polling* HTTP, ya que reduce la carga de la red y garantiza que eventos como "RECURSOS ASIGNADOS" o "DEADLOCK DETECTADO" fluyan al instante, ofreciendo una experiencia altamente reactiva en la interfaz de Vue.js.